



UNIVERSITÉ
**PARIS
SCIENCES
& LETTRES**

Statistical Learning II

Lecture 2 - Supervised learning

Bruno Loureiro
@ CSD, DI-ENS & CNRS

brloureiro@gmail.com

Supervised learning

Supervised Learning



Not grumpy



Grumpy

Supervised Learning



Not grumpy



Grumpy



???

Supervised Learning

Inputs / covariates $x \in \mathcal{X}$



x_1



x_2



x_3



y_1

Not grumpy



y_2

Grumpy



y_3

???

Outputs / Labels / response $y \in \mathcal{Y}$

Supervised Learning

Inputs / covariates $x \in \mathcal{X}$



x_1



x_2



x_3



y_1

Not grumpy

y_2

Grumpy

y_3

???

Outputs / Labels / response $y \in \mathcal{Y}$

Supervised Learning

Inputs / covariates $x \in \mathcal{X}$



x_1



x_2



x_3

$f: \mathcal{X} \rightarrow \mathcal{Y}$



y_1

Not grumpy

y_2

Grumpy

y_3

???

Outputs / Labels / response $y \in \mathcal{Y}$

Supervised Learning

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the training data.

Supervised Learning

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the **training data**.



- The **input space** $\mathcal{X}$ is often assumed to be a **vector space** $\mathcal{X} \subset \mathbb{R}^d$. But keep in mind in real life it can be any **data structure** (e.g. a pandas.DataFrame)

Supervised Learning

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the **training data**.



- The **input space** $\mathcal{X}$ is often assumed to be a **vector space** $\mathcal{X} \subset \mathbb{R}^d$. But keep in mind in real life it can be any **data structure** (e.g. a pandas.DataFrame)
- The **output space** $\mathcal{Y}$ is often assumed to be a subset $\mathcal{Y} \subset \mathbb{R}$.

Supervised Learning

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the **training data**.



- The **input space** $\mathcal{X}$ is often assumed to be a **vector space** $\mathcal{X} \subset \mathbb{R}^d$. But keep in mind in real life it can be any **data structure** (e.g. a pandas.DataFrame)
- The **output space** $\mathcal{Y}$ is often assumed to be a subset $\mathcal{Y} \subset \mathbb{R}$.
- In particular, if $|\mathcal{Y}| = k$ is a **discrete** set, we say we have a **classification problem**.

Supervised Learning

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the **training data**.



- The **input space** $\mathcal{X}$ is often assumed to be a **vector space** $\mathcal{X} \subset \mathbb{R}^d$. But keep in mind in real life it can be any **data structure** (e.g. a pandas.DataFrame)
- The **output space** $\mathcal{Y}$ is often assumed to be a subset $\mathcal{Y} \subset \mathbb{R}$.
- In particular, if $|\mathcal{Y}| = k$ is a **discrete** set, we say we have a **classification problem**.
- If $\mathcal{Y}$ is a **continuous** set, we say we have a **regression problem**.

Supervised Learning

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the **training data**.



- The **input space** $\mathcal{X}$ is often assumed to be a **vector space** $\mathcal{X} \subset \mathbb{R}^d$. But keep in mind in real life it can be any **data structure** (e.g. a pandas.DataFrame)
- The **output space** $\mathcal{Y}$ is often assumed to be a subset $\mathcal{Y} \subset \mathbb{R}$.
- In particular, if $|\mathcal{Y}| = k$ is a **discrete** set, we say we have a **classification problem**.
- If $\mathcal{Y}$ is a **continuous** set, we say we have a **regression problem**



It is very common to consider a **one-hot encoding** $\mathcal{Y} = \{0, 1\}^k$ in classification.

Supervised Learning

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the training data.

Examples of classification:

- Grumpy vs. Non-grumpy cats

$\mathcal{X} = \{\text{photos of cats}\}, \mathcal{Y} = \{\text{grumpy, not grumpy}\}$

- E-mail spam detection

$\mathcal{X} = \{\text{your inbox}\}, \mathcal{Y} = \{\text{spam, not spam}\}$

- Medical diagnosis

$\mathcal{X} = \{\text{medical data}\}, \mathcal{Y} = \{\text{diseases}\}$

- Sentiment analysis

$\mathcal{X} = \{\text{text}\}, \mathcal{Y} = \{\text{positive, negative, neutral}\}$

Supervised Learning

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the **training data**.

Examples of regression:

- Temperature prediction

$$\mathcal{X} = \mathbb{R}^3, \mathcal{Y} = \mathbb{R}$$

- Stock price prediction

$$\mathcal{X} = \{\text{list of stocks}\}, \mathcal{Y} = \mathbb{R}_+$$

- Life expectancy

$$\mathcal{X} = \{\text{medical data}\}, \mathcal{Y} = \mathbb{R}_+$$

- Any price, cost, income, etc. prediction.

Supervised Learning

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the training data.

Supervised Learning

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the training data.

In supervised learning, our goal is to use the data to learn a function that correctly assigns the labels to the responses.

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

Supervised Learning

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the **training data**.

In supervised learning, our goal is to use the data to **learn** a function that correctly **assigns the labels** to the **responses**.

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$



For **classification**, it is common to define instead:

$$f : \mathcal{X} \rightarrow [0,1]^{|\mathcal{Y}|}$$

Where $f(x)$ is a vector of **class probabilities**. In this case, final prediction is given by:

$$\hat{y} = \operatorname{argmax}_{k \in |\mathcal{Y}|} f(x)$$

Supervised Learning

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the training data.

In supervised learning, our goal is to use the data to learn a function that correctly assigns the labels to the responses.

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

Two key words: correctly and learn.

Loss function

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the **training data**.

In supervised learning, our goal is to use the data to **learn** a function that correctly **assigns the labels** to the **responses**.

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

Two key words: **correctly** and **learn**. To quantify the first, it is common to introduce a **loss function**:

$$\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$$

Loss function

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the **training data**.

In supervised learning, our goal is to use the data to **learn** a function that correctly **assigns the labels** to the **responses**.

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

Two key words: **correctly** and **learn**. To quantify the first, it is common to introduce a **loss function**:

$$\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$$

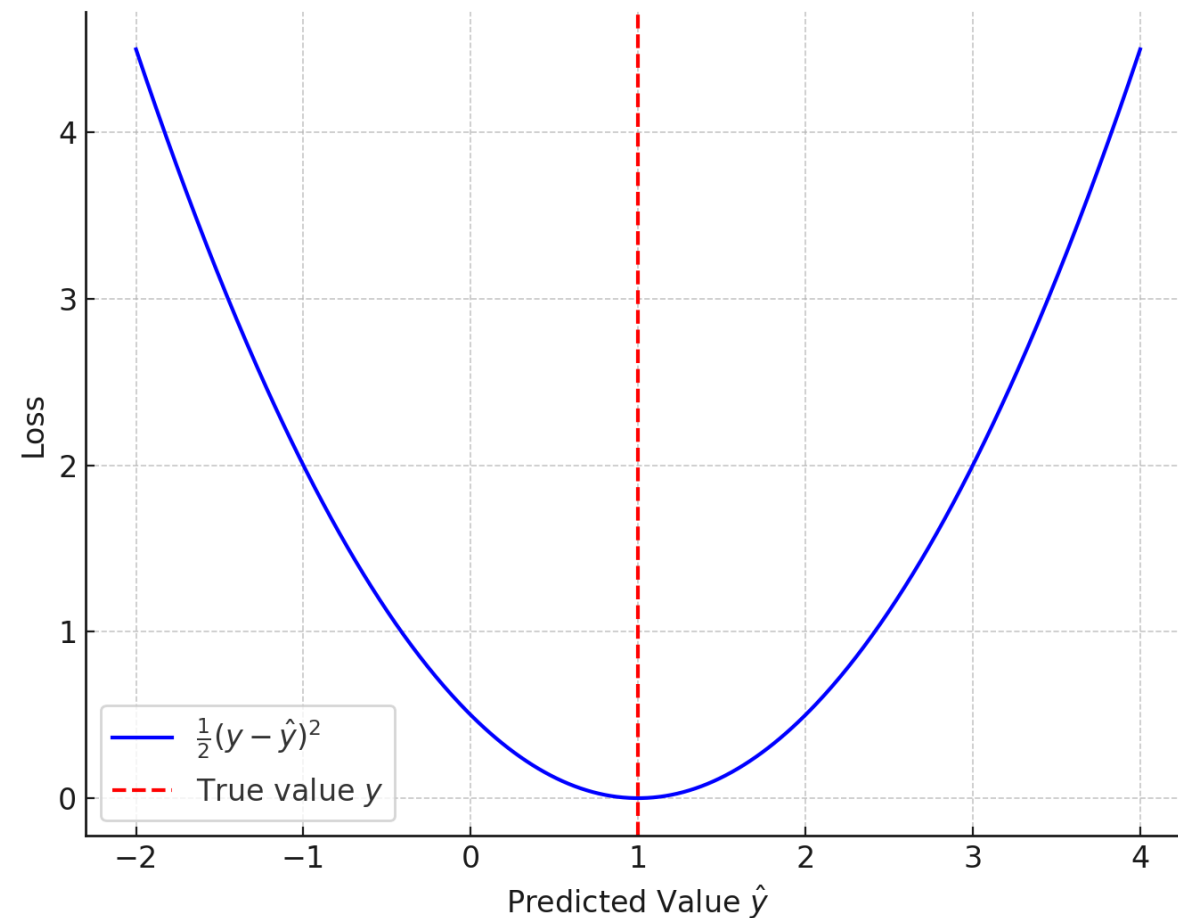


For **classification** this will also depend on the **encoding**.

Regression losses

Examples in regression:

- Square loss: $\ell(y, z) = \frac{1}{2}(y - z)^2$



Regression losses

Examples in regression:

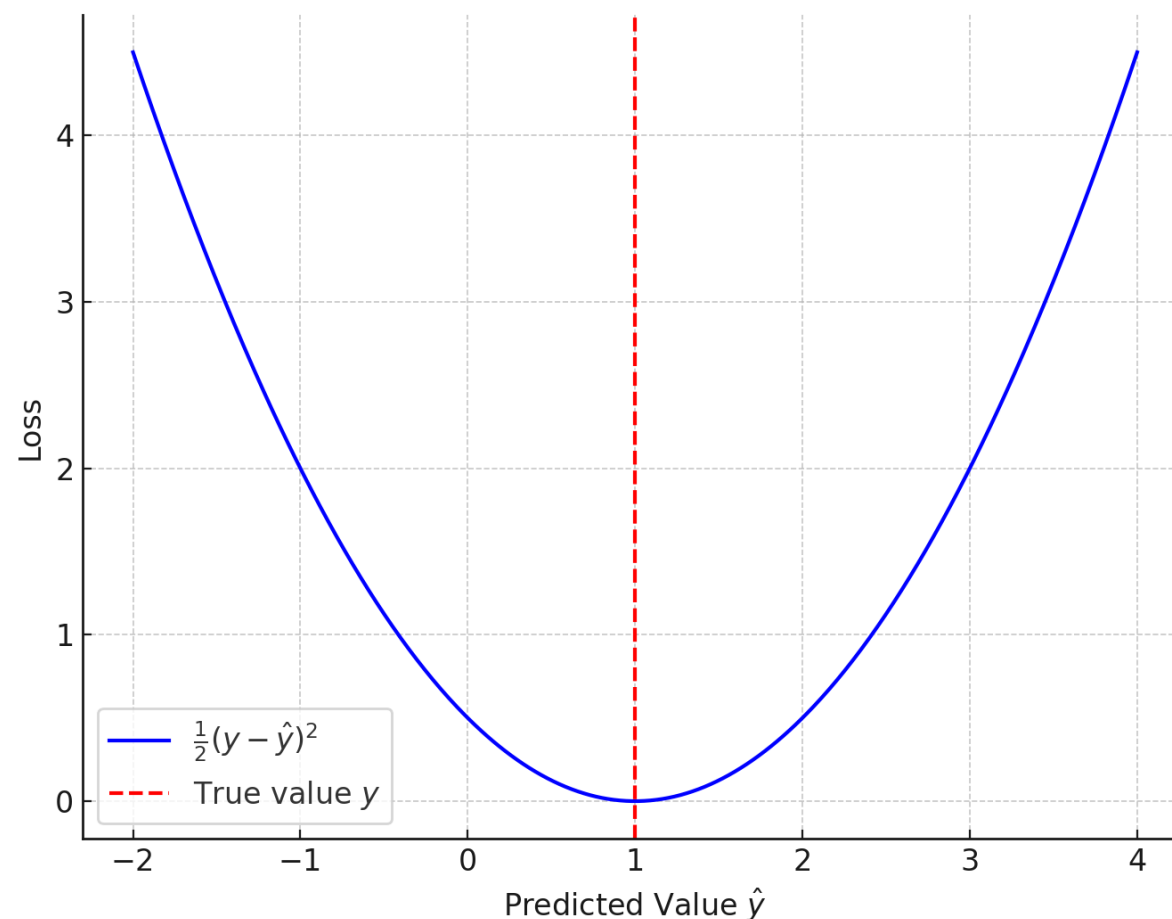
- Square loss: $\ell(y, z) = \frac{1}{2}(y - z)^2$



The square loss is sensitive to outliers



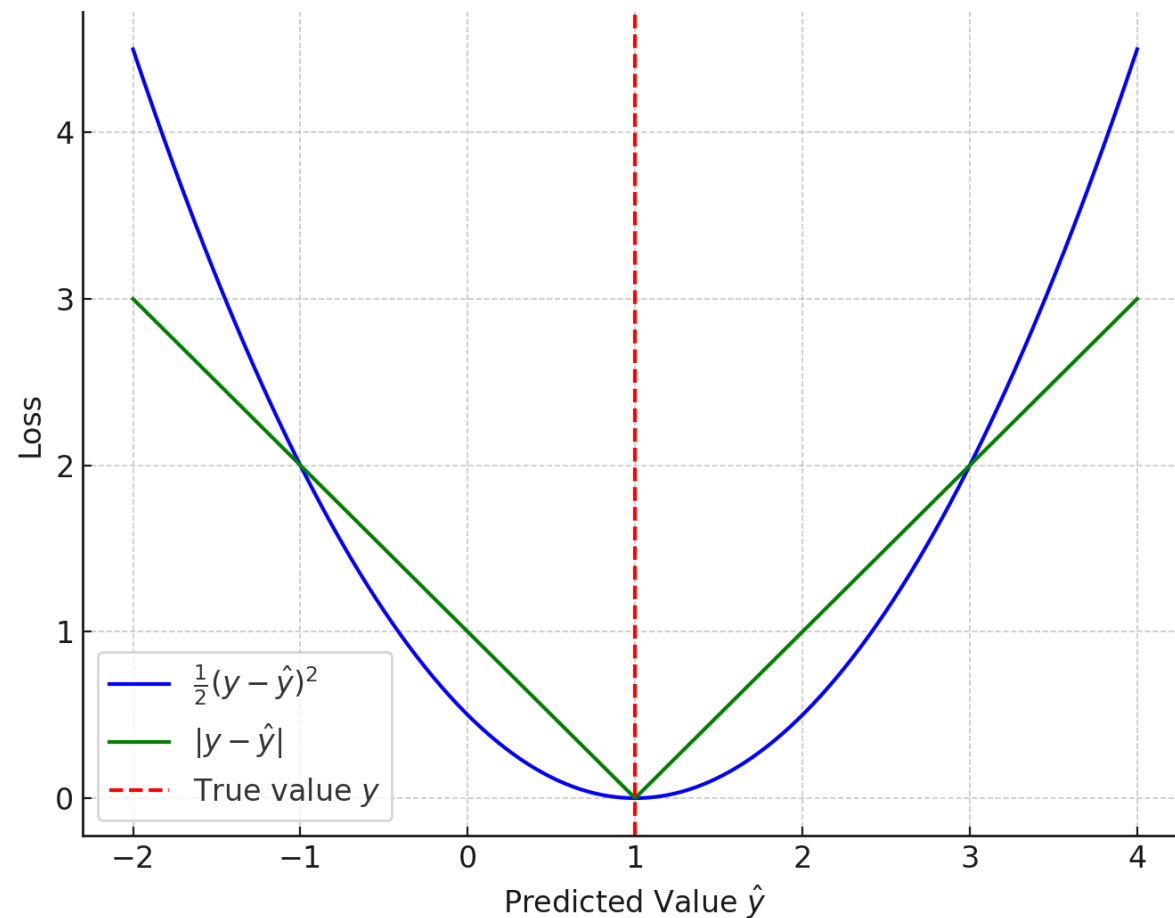
Exercise:
show this.



Regression losses

Examples in regression:

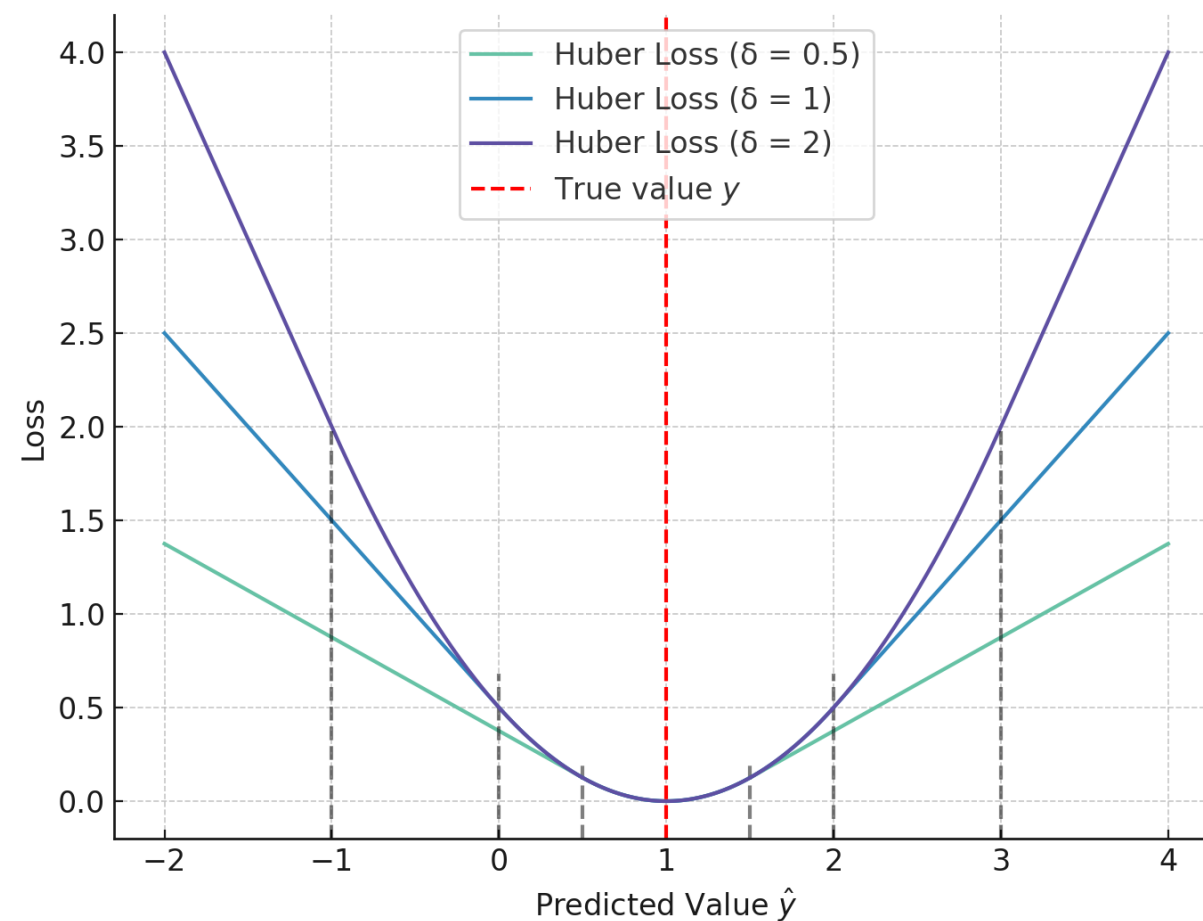
- Square loss: $\ell(y, z) = \frac{1}{2}(y - z)^2$
- Absolute loss: $\ell(y, z) = |y - z|$



Regression losses

Examples in regression:

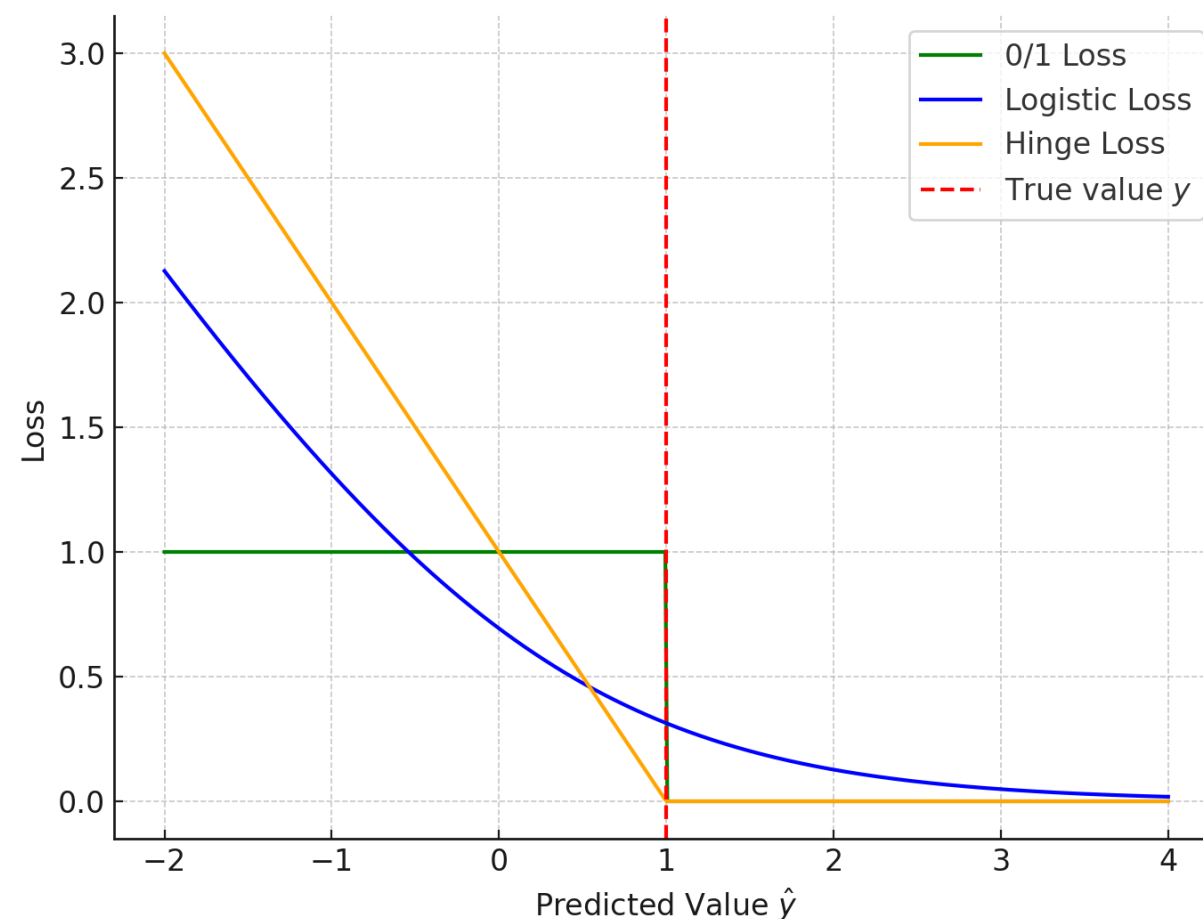
- Huber loss: $\ell_{\delta}(y, z) = \begin{cases} \frac{1}{2}(y - z)^2 & \text{if } |y - z| \leq \delta \\ \delta(|y - z| - \frac{1}{2}\delta) & \text{if } |y - z| > \delta \end{cases}$



Classification losses

Examples in binary classification $\mathcal{Y} = \{-1, +1\}$:

- 0/1 loss: $\ell(y, z) = \delta_{yz}$ (or $\ell(y, z) = \theta(y - z) = \begin{cases} 1 & \text{if } y - z \leq 0 \\ 0 & \text{otherwise} \end{cases}$)
- Logistic loss: $\ell(y, z) = \log(1 + e^{-yz})$
- Hinge loss: $\ell(y, z) = \max(0, 1 - yz)$



Empirical risk

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the **training data**.

Given a loss function $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$, and a predictor $f : \mathcal{X} \rightarrow \mathcal{Y}$ define the **empirical risk**:

$$\hat{\mathcal{R}}_n(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i))$$

Also known as the **training loss**.

Empirical risk

Let $\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \dots, n\}$ denote the **training data**.

Given a loss function $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$, and a predictor $f : \mathcal{X} \rightarrow \mathcal{Y}$ define the **empirical risk**:

$$\hat{\mathcal{R}}_n(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i))$$

Also known as the **training loss**. This quantifies how well we fit the data. But is this a good notion of learning?

$$f(x) = \begin{cases} y_i & \text{if } x \in \mathcal{D} \\ 0 & \text{otherwise} \end{cases} \quad \Rightarrow \quad \hat{\mathcal{R}}_n = 0$$